Unit1操作系统结构

操作系统自下而上可大致分为4部分：硬件、操作系统、应用程序、用户
操作系统是计算机系统中最基本的系统软件
并发：一段时间内同时执行/并行：同时进行运算或操作的特性
操作系统提供两类用户接口：
命令接口：联机命令接口(交互式命令接口)&脱机命令接口(批处理命令接口)
程序接口：由一组系统调用组成

操作系统发展历程：手工->脱机->批处理(单道批处理/多道批处理)->分时操作系统->实时操作系统->网络操作系统/分布式计算机系统->个人计算机操作系统

操作系统的基本特征包括：
并发/并行
共享：互斥共享方式&同时访问方式(微观也是轮流)
虚拟：隐藏实际实现的复杂性
异步：资源竞争导致的不确定性

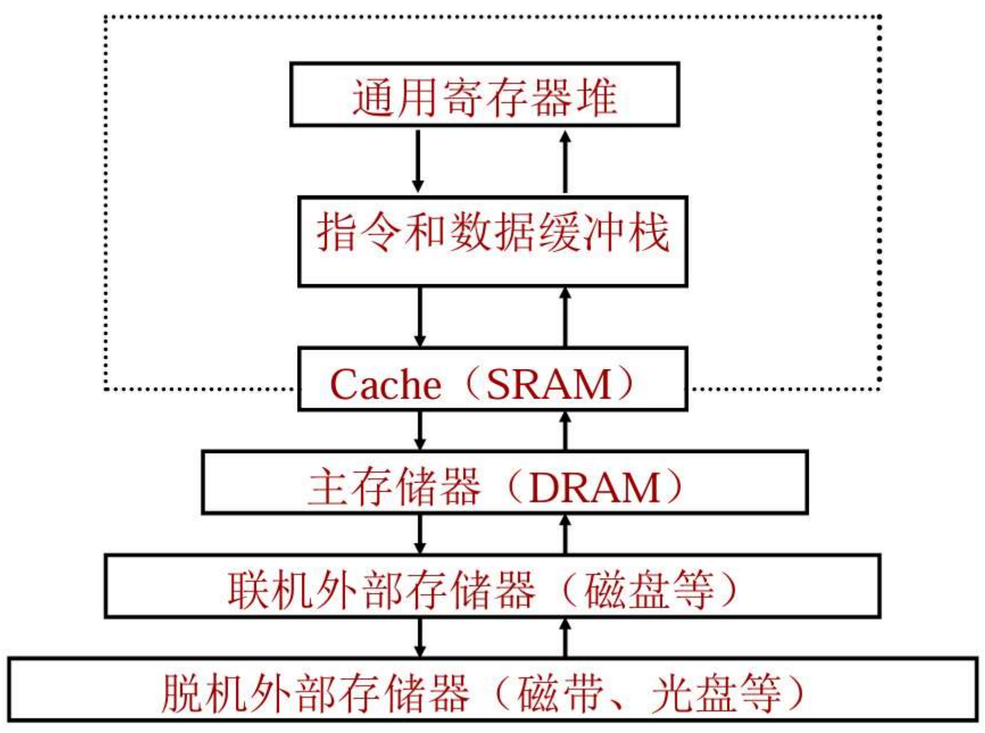


CPU运行模式分为用户态和内核态，分别执行非特权指令和特权指令
应用程序通过非特权指令-访管指令向操作系统请求服务
系统调用类型：
进程控制/文件管理/设备管理/信息维护/通信

	宏内核	微内核
基本概念	用户服务和内核服务运行在相同的地址空间中	用户服务和内核服务运行在不同的地址空间中
尺寸	比微内核大	较小
执行速度	快	慢
可扩展性	不容易扩展	容易扩展
安全性	单个服务崩溃往往意味着整个系统崩溃	单个服务崩溃不影响全局
开发代码量	代码量小	代码量大

操作系统启动过程：
① 加载操作系统：
从磁盘加载至内存特定位置(0x8000_0000)
②执行简易BIOS: 初始化函数栈、机器态初始化(CPU特权级等)
③执行XV6代码：初始化操作系统内存、虚拟内存、进程、磁盘模块，加载第一个用户进程
④开始进程调度：正式运行第一个用户进程，操作系统启动完毕，开始加载并执行其他程序

操作系统三大问题：虚拟化(内存管理、虚拟内存)/并发化(并发、同步、CPU调度)/持久化(IO、存储、文件系统)





Unit2进程

进程提供给应用程序的两个抽象：
逻辑控制流：通过内核上下文切换机制，每个程序似乎独占CPU
私有地址空间：内核的虚拟内存机制，每个程序似乎独占内存系统

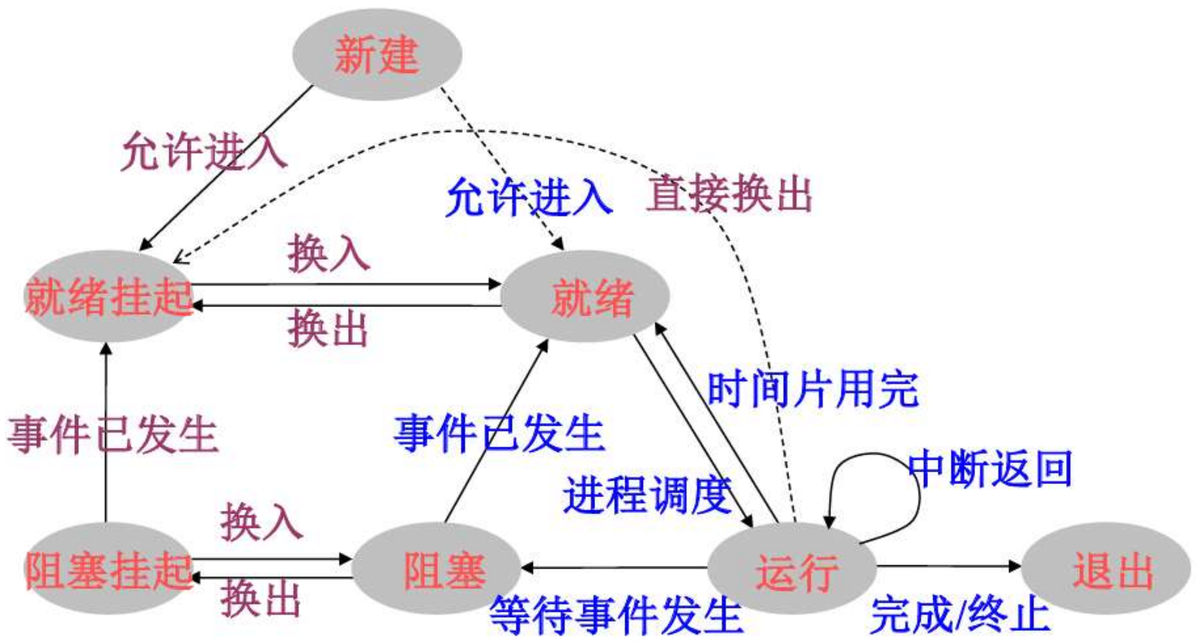
程序是一组指令的有序集合
通过多次执行，一个程序可对应多个进程；通过调用关系，一个进程可执行多个程序。

然而同一时刻一个进程只能执行一个程序

进程的五个基本特征：动态性/并发性/独立性/异步性/结构性
独立性：进程是os进行资源分配和调度的基本单位
结构性：进程=程序+数据+进程控制块PCB=进程上下文+代码+数据+栈=线程+代码+数据+内核上下文？->**不用记，纯纯糊涂账**

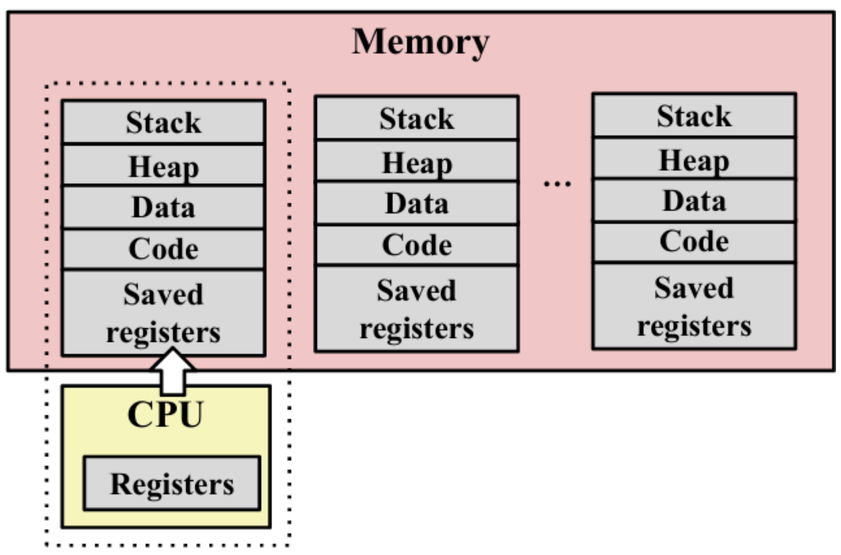
进程的五个状态:(前三个是基本状态)

就绪：获得了除CPU以外所需的一切资源
运行：在CPU执行
阻塞：也称等待态，是一种主动行为
新建：至少建立了PCB
终止：终止且等待父进程/系统回收资源

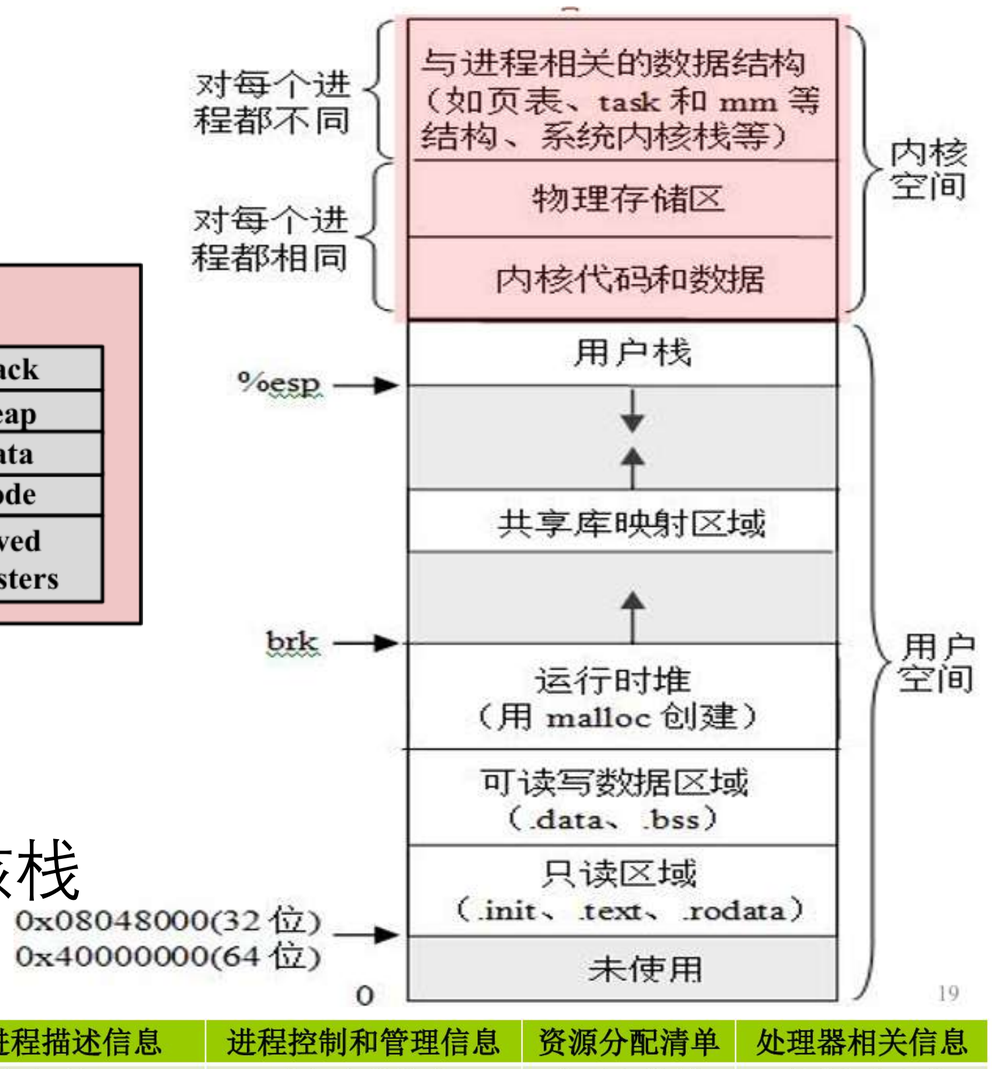


挂起转换：即主存和外存的交换

进程的上下文：
进程物理实体(代码及数据)+
进程运行的环境



进程的存储器映像：
用户级上下文地址空间+
系统级上下文空间



系统级上下文：
进程标识信息+进程现场信息+进程控制信息+系统内核栈
用户级上下文：
程序块+数据块+用户堆栈(堆+用户栈)

进程描述信息	进程控制和管理信息	资源分配清单	处理器相关信息
进程标识符（PID）	进程当前状态	代码段指针	通用寄存器值
用户标识符（UID）	进程优先级	数据段指针	地址寄存器值
	代码运行入口地址	堆栈段指针	控制寄存器值
	程序的外存地址	文件描述符	标志寄存器值
	进入内存时间	键盘	状态字
	处理机占用时机	鼠标	
	信号量使用		

进程现场信息：用于存储被换下的寄存器上下文

PCB
位于系统级上下文，描述进程与其他进程、系统资源的**关系**以及进程在各个不同时期所处的**状态**的数据结构
包含：进程描述信息(PID&UID)/进程控制和管理信息/资源分配清单/处理器相关信息（即包含上面的进程标识信息、控制信息，部分现场信息）

内核：一块共享的内存驻留的操作系统代码
进程切换时执行的就是内核的代码

僵尸进程：终止了但还未被回收
如果父进程没有回收子进程并终止，则僵尸进程将会由Init进程(PID=1)进行回收
孤儿进程：即使父进程被终止，子进程仍然可以依然活跃，必须显式杀死子进程，否则它将会永远运行。孤儿进程也会成为init的子进程

Fork
调用者: 返回子进程的PID或-1(error)；新建的子进程: 返回0
Exit
正常结束时返回 0，错误时返回非0值
只会被调用一次，并且不会再次返回
另一种显式设置退出状态的方法是从主程序返回一个整数值
Wait　　pid_t wait(int *status);
返回值是子进程PID，多个子进程几乎同时退出时，内核会从已经结束的子进程中任意选择一个返回给父进程　　pid_t waitpid(pid_t pid, int *status, int options=0);
Waitpid
等待特定进程
Sleep
正常退出返回0，被信号中断返回剩余休眠时间/秒
Pause
进入睡眠状态，直到进程接到信号为止
Execve
加载并运行新程序，替换当前进程的代码、数据、堆栈，不创建新进程，即保留PID、打开文件和上下文信号
参数列表argv，通常argv[0]=filename可执行文件
调用一次并且永不返回，发生错误则抛出异常

Fork+execve共同完成一个新进程的完整创建

CPU的控制流：一个指令的运行序列，每个进程都是逻辑控制流
异常控制流：低层次机制：软硬件组合实现对系统的时间做出响应
高层次机制：进程上下文切换/信号/全局跳转
异常（Exception）：响应某些事件（即处理器状态更改）进而将控制权转移到OS内核

类别	原因	异步/同步	返回行为
中断（Interrupt）	来自I/O设备的信号	异步	总是返回到下一条指令
陷阱（Trap）	有意的异常	同步	总是返回到下一条指令
故障（Fault）	潜在的可恢复的错误	同步	可能返回到当前指令
终止（Abort）	不可恢复的错误	同步	不会返回

进程间通信:
常用的通信类型:
共享内存系统 (Shared-Memory System) /消息传递系统 (Message passing System) /管道(pipe)/套接字/远程过程调用

共享内存系统：没有同步机制，申请一个共享存储分区映射到不同进程的虚拟地址空间中；用完后先释放再删除

消息传递系统：计网的报文，收发双方通过明确的协议和消息格式来通信，格式包括消息头和消息正文
消息缓冲队列(直接通信)：发送进程直接将消息发送给接收进程，并将它挂在接收进程的消息缓冲队列上。接收进程从消息缓冲队列中取得消息
信箱通信(间接通信)：发送进程将消息发送到某个中间实体（一般称为信箱）中，接收进程从中取得消息

管道：命名管道-任意进程间通信
普通管道机制:
单根管道只能半双工通信，fd[0]为读出端，fd[1]为写出端
三个要求：互斥(半双工)，同步(写端写后睡眠直到数据读出，读端读到空时也睡眠直到写入)，只有确定对方存在才使用管道通信

管道详解:
创建：pipe(p) - 在内核创建缓冲区，返回两个fd
初始状态（fork后）：
父进程：拥有 p[0]（读端），p[1]（写端）
子进程：拥有 p[0]（读端），p[1]（写端）
每个文件描述符的引用计数都为2

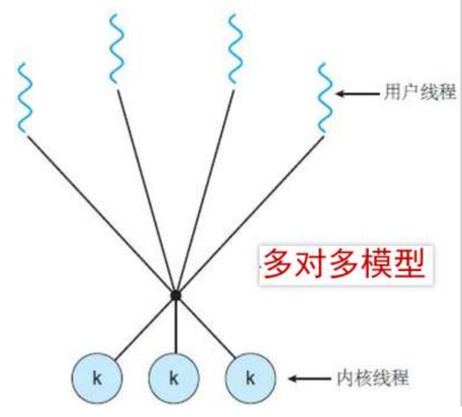
各自关闭不需要的一端:
子进程：close(p[1]) → 写端引用计数减为1
父进程：close(p[0]) → 读端引用计数减为1

数据传输完成后关闭:
父进程写入后 close(p[1]) → 写端引用计数变为0
子进程读取后 close(p[0]) → 读端引用计数变为0
当两端都关闭后，管道资源被系统回收

当所有写端都被关闭时，读端会收到EOF（返回0）
当所有读端都被关闭时，写端若还写会收到SIGPIPE信号（默认导致进程终止）

死锁风险：如果子进程不关闭写端，父进程写入后关闭写端，但子进程的写端还开着，父进程会等待所有写端关闭，而子进程在等待数据，形成死锁

Unit3并发与同步



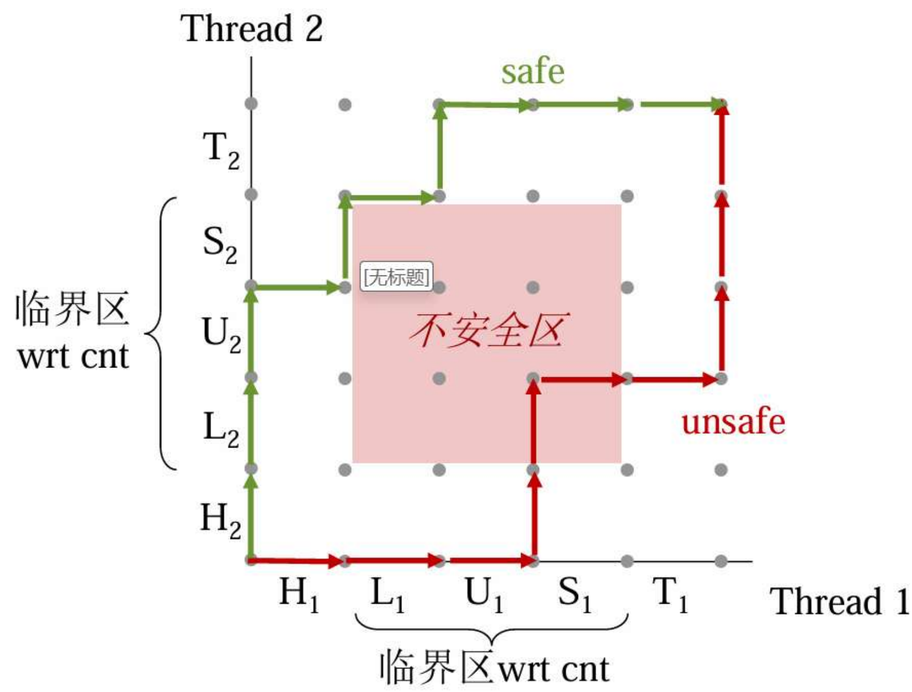
线程Thread-CPU执行和调度的基本单位
基本状态：就绪态/运行态/阻塞态
共享的：所属进程的地址空间
独有的：局部变量的堆栈，线程TID，线程上下文
两种支持方法：用户级线程/内核级线程
用户级线程和内核级线程之间关系：多对一模型 /一对一模型 /多对多模型
多对多变体：双层模型-也允许将一个用户线程绑定一个内核线程

线程API-POSIX线程
int pthread_create(pthread_t *tid, pthread_attr_t *attr, func *f, void *arg)
tid： 返回创建的新线程的pid
attr（属性）： 设置线程的属性
f： 创建线程后将执行的程序
arg（参数）： 一个可以传递给开始程序的参数或者包含多个参数的结构体。 它必须作为空类型的指针强制转换通过引用传递。 如果不传递任何参数，则可以使用NULL
传参经典错误
(void *) &t传入了t的地址，线程运行时才去地址找值，可能已经改变
(void *) taskids+t，tasked+t是一个值，向不同线程传递了不同的值，按照不同的值去寻址
void pthread_exit(void *thread_return)
此为显式终止线程，若高一级进程返回时则线程会隐式终止
主线程调用的话会自动等待所有其他线程终止后终止，返回值通过thread_return传出
int pthread_cancel(pthread_t tid)
请求线程取消。直到到达某个取消点才终止。取消点是线程检查是否被取消并按照请求进行动作的一个位置。
joinable状态(默认)和unjoinable状态，前者退出时不会释放占用的堆栈和线程描述符，被join才会释放
int pthread_join(pthread_t tid, void **thread_return)
阻塞线程直到tid线程终止，对同一个线程尝试多个join操作会发生逻辑错误
int pthread_detach(pthread_t tid)
函数将线程 tid 分离出来，其结束后自动回收资源。一经分离，不可再被join

变量访问权限：
[全局变量]在main函数之外创建的全局变量a可以被主线程及所有子线程访问
[本地静态变量]子线程共同调用的函数func1创建的static变量b，可以被各个子线程访问但不能被主线程访问
[本地自动变量]主线程创建的局部变量c，只能被主线程直接访问，但是子线程可以通过全局指针变量a间接访问
[本地自动变量]子线程创建的局部变量d，只能被他自己访问

变量共享：当且仅当它的一个实例被一个以上的线程引用

线程进度图:
右图红色为H1, L1, U1, H2, L2, S1, T1, U2, S2, T2
轨迹（trajectory）是一系列合法 的状态转换
一个轨迹是安全的当且仅 当它没有进入不安全区
一个轨迹是正确的当且 仅当它是安全的
操作共享变量的指令LUS组成关于共享变量cnt的临界区
两个临界区的交集形成的状态空间称为不安全区



非负信号量s
原子操作P(s):wait
s>0则s--或s=0则阻塞进程
被重启应该执行s--(因为其直接进入阻塞还没有执行过--)
原子操作V(s):signal
s++
存在阻塞的P则重启其中一个

允许负的信号量s
原子操作P(s):wait
s--
s<0则阻塞
被唤醒时不需要再s--
原子操作V(s):signal
s++
s<=0则换新一个阻塞在s上的进程 大于0说明当前没有阻塞在s的线程

POSIX给出的接口
int sem_init(sem_t *s, int pshared, unsigned int val);}
s为指向信号量结构的一个指针； pshared不为 0 时此信号量在进程间共享，否则只能为当前进程的所有线程共享； val给出了信号量的初始值。 成功返回0，错误返回-1
int sem_wait(sem_t*s); //P(s)
int sem_post(sem_t*s); //V(s)

互斥锁：s仅允许为0或者1

进程间制约关系
间接制约：临界资源&互斥(访问是无序的)
直接制约：相互协作(保证同步有序访问)

同步的准则:
空闲让进：其他进程均不处于临界区/**忙则等待**：已有进程处于其临界区/**有限等待**：等待进入临界区的进程不能"死等“/**让权等待**：不能进入临界区的进程，应释放CPU（如转换到等待状态）

两个经典问题

生产者-消费者问题(经典同步)

■插入一个项目（生产者）：

```
/* 在缓冲区尾插入一个项目 */
void sbuf_insert(sbuf_t *sp, int
item)
{    /* 等待可用槽位*/
     P(&sp->slots);
     /* 给buffer加锁*/
     P(&sp->mutex);
     /* 插入项目*/
     sp->buf[(++sp->rear)%(sp->
n)] = item;
     /* 解锁buffer*/
     V(&sp->mutex);
     /* 宣告这里有可用的项目 */
     V(&sp->items);
}                                  sbuf.c
```

■移除一个项目（消费者）：

```
/* 移除并返回第一个项目 */
int sbuf_remove(sbuf_t *sp)
{    int item;
     /* 等待可用项目*/
     P(&sp->items);
     /* 给buffer加锁*/
     P(&sp->mutex);
     /* 移除项目 */
     item = sp->buf[(++sp->
front)%(sp->n)];
     /* 解锁buffer*/
     V(&sp->mutex);
     /* 宣告这里有可用的槽位 */
     V(&sp->slots);
     return item;}
```

读者写者问题(经典互斥) 互斥锁可用于多进程或多线程之间，但只能由对它加锁的线程或进程来解锁

第一类读者-写者问题（读者优先） 要求不让读者等待，除非已经把共享对象的使用权限给了一个写者，一个后来的读者也比前面的写者优先级高

Readers:

```
int readcnt;     /* Initially = 0 */
sem_t mutex, w; /* Initially = 1 */
void reader(void)
{
    while (1) {
       P(&mutex);
       readcnt++;
       if (readcnt == 1) /* 第一个进入 */
            P(&w);
       V(&mutex);

       /* 这里放临界区代码 */
       /* 读 */
       P(&mutex);
       readcnt--;
       if (readcnt == 0) /* 最后一个离开 */
            V(&w);
       V(&mutex);
    }
}
```

Writers:

```
void writer(void)
{
    while (1) {
       P(&w);

       /* 这里放临界区代码 */
       /* 写 */

       V(&w);
    }
}                                  rw1.c
```

只有第一个读者获取锁w，最后一个释放w，中间的读完全不需要等待w，因此可以插队写者

第二类读者-写者问题（写者优先） 当一个写者准备好要写，它就会尽可能快地完成写操作，在一个写者后到达的读者必须等待，即使这个写者也在等待

```
int writecnt;     /* Initially = 0 */
sem_t mutex, r, b; /* Initially = 1 */
void writer(void)
{
    while (1) {
       P(&mutex);
       writecnt++;
       if (writecnt== 1) /* 第一个进入 */
              P(&r);
       V(&mutex);

       P(&B);
       /* 这里放临界区代码 */
       /* 写 */
       V(&B);

       P(&mutex);
       writecnt--;
       if (writecnt== 0) /* 第一个离开 */
              V(&r);
       V(&mutex);
    }
}
```

Writers

```
void reader(void)
{
    while (1) {
       P(&r);

       /* 这里放临界区代码 */
       /* 读 */

       V(&r);
    }
}
```

Readers

然而这个版本不支持并发读

第二类-支持并发读：

```
int writecnt;     /* Initially = 0 */
sem_t mutex, r, B; /* Initially = 1 */
void writer(void)
{   while (1) {
        P(&mutex2);
        writecnt++;
        if (writecnt== 1) /* 第一个进入 */
                P(&S);
        V(&mutex2);

        P(&wmutex);
        /* 这里放临界区代码 */
        /* 写 */
        V(&wmutex);

        P(&mutex2);
        writecnt--;
        if (writecnt== 0) /* 第一个离开 */
                V(&S);
        V(&mutex2);
    }
}
```

Writers

```
void reader(void)
{
    while (1) {
        P(&S2); P(&S);
        P(mutex1);
        readcnt++;
        if (readcnt == 1) /* 第一个进入*/
            P(&wmutex);
        V(&mutex1);
        V(&S); V(&S2);

        /* 这里放临界区代码 */
        /* 读 */

        P(&mutex1);
        readcnt--;
        if (readcnt == 0) /* 最后一个离开 */
            V(&wmutex);
        V(&mutex1);
    }
}
```

Readers

哲学家问题-三种解决方案

至多只允许有四位哲学家同时去拿左边的筷子

仅当哲学家的左、右两支筷子可用时，才允许他拿起筷子；

规定奇数号哲学家先拿他左边的筷子，然后再去拿右边的筷子；而偶数号哲学家相反。

```
semaphore chopstick[0…4]={1,NULL};
semaphore sm={4,NULL};

program philosopher(i){
    P(sm);
    P(chopstick[i]);
    P(chopstick[(i+1)mod 5]);
    eating;
    V(chopstick[i]);
    V(chopstick[(i+1)mod 5]);
    V(sm);
}
```

线程安全：

多个并发线程反复调用时，它会一直产生正确的结果

Class 1: 不保护共享变量的函数

Class 2: 保持跨越多个调用的状态的函数

Class 3: 返回指向静态变量的指针的函数

Class 4: 调用线程不安全函数的函数

线程安全函数的子集-可重入函数

不引入任何共享数据，可以被打断重新调用，不依赖锁，可被一个线程反复调用

大多数Linux函数，包括定义在标准C库中的函数都是线程安全的

- 例如: malloc, free, printf, scanf

只有一小部分是例外，例如:

Thread-unsafe function	Class	Reentrant version
asctime	3	asctime_r
ctime	3	ctime_r
gethostbyaddr	3	gethostbyaddr_r
gethostbyname	3	gethostbyname_r
inet_ntoa	3	(none)
localtime	3	localtime_r
rand	2	rand_r

信号

ID Name	默认操作	对应的事件
2 SIGINT	终止Terminate	用户按下 ctrl+c
9 SIGKILL	终止Terminate	杀死程序（Kill program，不可否决或忽略）
11 SIGSEGV	终止Terminate	无效的内存引用
14 SIGALRM	终止Terminate	计时器信号
17 SIGCHLD	忽略Ignore	子进程终止运行

1.信号的产生可以是内核也可以是进程，信号由内核发送，方式是改变进程上下文的某些状态
2.进程可以自己给自己发信号
3.当内核决定传递信号给进程时，有以下几种处理方式：
sighandler_t signal(int signum, sighandler_t handler)
忽略(handler:SIG_IGN)：内核直接清除信号位，无事发生。
默认操作(handler: SIG_DFL)：内核执行预设动作（终止、停止、忽略、转储核心等）。
捕获(handler:用户注册的处理函数)：执行一个用户级的信号处理程序（signal handler）来接收信号
4.信号是内核异步打断进程的执行流，而中断是异步打断CPU
5.对一个进程而言，每个类型的信号最多只能有一个待处理信号，同类信号不会排队，后来者丢弃。一个待处理信号最多只能被接收一次
6.进程可以选择性地阻塞某些信号的接收；被阻塞的信号也可以传输，但在它被解除阻塞之前，该信号不能被接收
7.阻塞位向量&挂起位向量：每个ID对应1位，前者1表示被阻塞，后者在传输时置1，接收后清除；阻塞表示这个信号暂时不会被传递给进程，可由进程自己显式设置，也可进程自己解除；挂起表示信号产生但还未传递
8.嵌套的信号处理程序：信号处理程序可以被其他信号处理程序中断,然而内核默认阻塞正在处理的同类型待处理信号

进程通过调用kill函数向其他进程（包括它自己）发送信号
int kill(pid_t pid, int sig)；
pid>0：发送信号sig给进程pid
pid=0：发送信号sig给调用进程的进程组内包括自身的所有进程
pid<0：发送信号sig给进程组|pid|内的所有进程

Unit4-CPU调度

调度时机分类调度:
非抢占式调度：任务主动让出
抢占式调度：os强制暂停并切换

三种任务类型：一般任务(相对短CPU区间多，区间长度越长，频次指数衰减)，IO约束型(短CPU区间更多)，CPU约束型(长CPU区间更多)
或两种任务类型：交互式/批处理

计算前核心概念:
吞吐量：单位时间完成的任务数量
周转时间：完成时间 - 提交时间/到达就绪队列时间
等待时间：任务在就绪队列等待的时间总和
响应时间：首次运行时间 - 提交时间/到达就绪队列时间

调度算法
先到先服务调度FCFS
最短作业优先调度SJF-不可抢占，只选队列里最短作为下一个
最短剩余作业优先调度SRJF-SJF的可抢占版本，在任务完成或有任务到来时，选择剩余时间最短的执行
N+1的预测时间公式：=> $\tau_{n+1} = \sum_{i=0}^{n} \alpha(1-\alpha)^{n-i} t_i + (1-\alpha)^{n+1}\tau_0$ α通常取0.5
SJF一般化=>优先权调度

轮询调度算法RR-按时间片(一般时间片10-100ms)以到达顺序轮询调度/切换时间0.1-1ms
注意，切换的时机是时间片用尽或者任务已经完成
切换操作实际上是将当前进程(如果未完成)放到当前就绪队列末尾，因此到达时间不同时也就知道怎么调度了
多级反馈队列MFQS-将任务按照优先级分为多个队列，每次从优先级最高的队列选择任务，使用太多CPU时间的任务会被放置在低优先级队列。在低优先级队列过久的任务会被移到高优先级队列(老化)
彩票算法-即按权重分配不同进程不同数量彩票，每次抽奖决定运行谁

Unit5-死锁

多个进程（线程）因循环等待资源而造成无法执行的现象
死锁四个必要条件:
互斥使用(Mutual exclusion) 至少有一个资源互斥使用，互斥即只可供一人用
不可抢占(No preemption) 资源只能自愿放弃
请求和保持(Hold and wait) 进程必须占有资源，再去申请
循环等待(Circular wait) 在资源分配图中存在一个环路

死锁的处理方法

死锁预防:

破除四条件:

互斥使用：通常无法破除

不可抢占：如果一个进程占有资源并申请另一个不能立即分配的资源，那么已分配资源就可被抢占/如果申请的资源得到满足，则抢占其他资源一次性分配给该进程/状态能保存和恢复的资源才能这么做

请求和保持：在进程执行前，一次性申请所有需要的资源

需要预知未来，编程困难/许多资源分配后很长时间后才使用，资源利用率低

破除循环等待：对资源类型进行排序，资源申请必须按序进行

死锁避免:

一个可完成的执行序列称安全序列，系统存在一个安全序列称安全状态

银行家算法:

Available：资源当前可用总量(只用于初始化work) Need进程还需要的资源数量 Work当前含可回收的可分配资源 Finish进程是否结束 Allocation进程已经获得的资源数量

■ 当前状态:

		Allocation	Need	Available
	Work=[3 3 2]	A B C	A B C	A B C
P_1	Work=[5 3 2]	~~P0 0 1 0~~	~~7 4 3~~	3 3 2
P_3	Work=[7 4 3]	~~P1 2 0 0~~	~~1 2 2~~	
P_4	Work=[7 4 5]	~~P2 3 0 2~~	~~6 0 0~~	
P_0	Work=[7 5 5]	~~P3 2 1 1~~	~~0 1 1~~	
P_2	Work=[10 5 7]	~~P4 0 0 2~~	~~4 3 1~~	

安全序列是<P_1, P_3, P_4, P_0, P_2>　　安全序列是唯一的吗？ 不唯一

循环遍历寻找安全序列O(mn^2)，n是线程总数，m是资源种类数

注意，当一个未finish的线程加入安全序列时，应该设置其finish=1

请求算法:

只要线程的请求request(必须小于need，否则报错)小于available就先分配，然后调用bank看当前有没有安全序列，无则拒绝请求并回滚

死锁检测+恢复:

请求算法基础上，不再每次调用bank检测，而是定时检测或者当发现资源利用率低时检测

相比银行家检测，这里面将未分配任何资源的对象也算作finish已结束(=1)，不再检测考虑；运行完遍历看还有没有finish=false，有就说明发生了死锁

死锁恢复：终止进程/回滚进程以剥夺资源

死锁忽略:

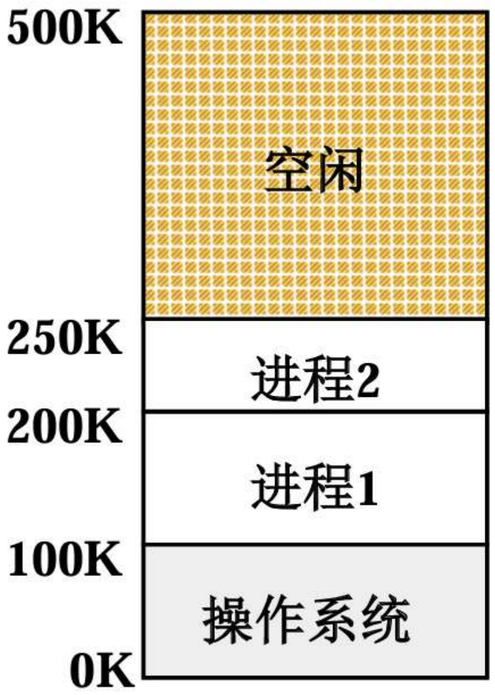
鸵鸟算法：对死锁不做任何处理，只能出现死锁时手动干预，如重启(大多数os选择)

Unit6-内存管理-解决的是程序如何使用内存的问题

源程序->内存中可执行程序：编译/链接/装入
装入方式：
绝对装入：提前直到程序放到哪儿，链接时直接产生绝对地址的绝对代码。
可重定位装入/静态重定位：编译链接后初始地址0，使用相对地址，装入时一次完成地址转换(重定位)
动态运行时装入/动态重定位：相比于静态，将重定位工作移到真正要执行时。

物理地址/虚拟地址/逻辑地址
虚拟地址是段名+段内偏移，逻辑地址是段内相对地址的部分，是虚拟地址的一部分
逻辑地址集合即逻辑地址空间/物理内存地址集合即物理地址空间/线性地址空间指CPU可以访问的所有地址集合，或者指虚拟地址空间

内存管理：
分区：此情况下进程的code、data、stack、heap作为同一整体进行分配
固定分区-等长分区：内存分为等大连续分区
固定分区-变长分区：内存分为大小不同连续分区，请求不超总大小的情况下，分配时找出能满足请求的最小分区，找不到就加入请求队列
可变分区：未使用的内存划为空闲分区，请求不超内存总大小情况下，如果能满足则划出请求的大小，否则加入请求队列等待。进程结束会重新标记空闲，因此空闲区不连续



空闲分区表

始址	长度
250K	250K

已分配分区表

始址	长度	标志
0K	100K	OS
100K	100K	P1
200K	50K	P2

外部碎片：像上面不断装入移出，内存中存在的小的不连续的不可用分区
内部碎片：固定分区中分配给某进程后剩余的分区
内存紧缩可以处理外部碎片但耗时巨大

可变分区-分段：解决堆栈动态增长问题
一个进程有一个段表，不同段分别存储在内存的不同位置
分段地址的翻译(逻辑地址汇编时从0开始编址，链接时可不为0)
逻辑地址->段表->物理地址
逻辑地址结构：seg|offset
根据seg查程序段表得到基址，基址+偏移=物理地址
Seg=逻辑地址整除逻辑段大小，offset=逻辑地址%逻辑段大小

分页：消除外部碎片，有限的内部碎片，每个进程一个页表
物理页号=帧号=页框号
页面尺寸通常4KB，32位机器=>20位编码页框号，12位编码页内偏移
地址翻译：
逻辑地址->页表项PTE->物理地址
逻辑地址结构：若干级页号|页内偏移
PTE结构：20位页框号|3位保留|9位信息(页号是隐含的)
N级页表访存n+1次
TLB-寄存器组，进一步提升速度
TLB命中也要访问一次内存

Read start
有些时候希望某些条目固定下来(如内核代码)，某些TLB的设计有这样 的功能，不被选择替换/进程切换后，所有的TLB表项都变为无效(flush)/如果进程马上又切换回来，则这种策略就很低效。有的TLB设计中条目项保存ASID(Address-space identifier)，此时不需要flush。
Read end

段页式管理：
逻辑地址结构：段号S|页号P|页内偏移量W
段表项结构：包含段号(隐含)、页表长度、页表始址
每个段有一张页表，至少包含页号和页框号

感觉不是重点
Intel x86内存管理硬件
实模式：1M-20位地址/段寄存器存储段基址
保护模式：4G-32位地址/开启分页/段选择器存储段描述符表中索引

Unit7-虚拟内存-制造连续内存的假象/制造比物理内存更大的空间/内外存交换

在内存管理中已经初见端倪，九位信息位中，
P--位0是存在(有效)标志。Present
R/W--位1是读/写标志。Accessed
A--位5是已访问标志。 Read/Write
D--位6是页面已被修改标志。Dirty
异常处理寄存器会记录缺页错误的指令地址，以便调页后重新执行

虚拟存储只能基于非连续分配技术

一级不缺页，要两次访存

缺页中断→决定淘汰页→页面调出→页面调入→重新访问页表(1次访存)→读数据(一次访存)

虚拟内存置换算法：FIFO/OPT/LRU
先进先出FIFO：淘汰最早调入的页面
最佳页面置换OPT：淘汰未来最远将使用的页
最近最少使用页面置换 LRU：淘汰最近最长时间没有使用的页

作业在运行前不必全部装入内存，且在运行过程中也不必一直驻留内存

LRU的两种准确实现

计数器法(代价太大)：初始为0，每次访问就加1，引用页时将计数器值复制到页表项中(这即记录了使用的时间)；置换时淘汰值最小的页(即最久远使用的)

计数器（每次访问，加1）	访问序列-> A	B	C	A	B	D	A	D	B	C	B
A	1	1	1	4	4	4	7	7	7	7	7
B	0	2	2	2	5	5	5	5	9	9	11
C	0	0	3	3	3	3	3	3	3	10	10
D	0	0	0	0	0	6	6	8	8	8	8

页码栈法(代价依然大)：

建立一个容量为有效帧数的页码栈。每当引用一个页时，该页号就从栈 中上升到栈的顶部，栈底为LRU页。当需要置换页时，直接置换栈底页

页码栈	A	B	C	A	B	D	A	D	B	C	B
			C	A	B	D	A	D	B	C	B
		B	B	C	A	B	D	A	D	B	C
	A	A	A	B	C	A	B	B	A	D	D

LRU近似实现-再给一次机会算法

CLOCK-算法-循环队列

A:如存在页命中，访问位置“1”，查询指针保持不动

B:否则，循环检查各页面的使用情况。

1、若访问位为“0”，选择该页淘汰，查询指针前进一步

2、若访问位为“1”，复位访问位为“0”，查询指针进一步

缺页较少时，clock退化为FIFO

CLOCK改进-引入额外指针

一个指针定期扫描清除“1“

间隔根据负载在fastscan~slowscan间变化

CLOCK改进-AM位判断

根据访问A和修改为M(或者是D)

第一轮：查找A=0,M=0页面，若找到，替换；否则，进入下一步

第二轮：查找A=0,M=1页面，若找到，替换；并把遍历过的页面的A位复位为“0”；若一直没找到，进入下一轮

第三轮：把所有页面的A位复位为“0”，重复第一轮，必要时再重复第二轮

其余相关问题

写时复制：子进程共享父进程的地址空间，只有要写某些页时才产生新的页作副本

交换空间：换下的页放到pagefile.sys文件(windows)，或生磁盘中(linux的swap分区)

工作集(驻留集)：给进程分配的主存物理空间，实际即物理页框数

固定分配：按照程序大小比例和优先权分配不同的固定大小的工作集

可变分配：工作集大小可变，缺页率越高，增大工作集

固定分配常搭配局部置换：只在本进程的工作集找牺牲页面

可变分配-全局置换：不考虑牺牲者属于哪个进程

可变也可用局部，但固定不能用全局

工作集过大则请求调页失去意义，过小则可能颠簸导致CPU率非常低，期望分配的工作集能覆盖一个局部的宽度

处理颠簸：
页错误率(PFF) = 页错误/指令执行条数，如果PFF>上限，增加分配帧数

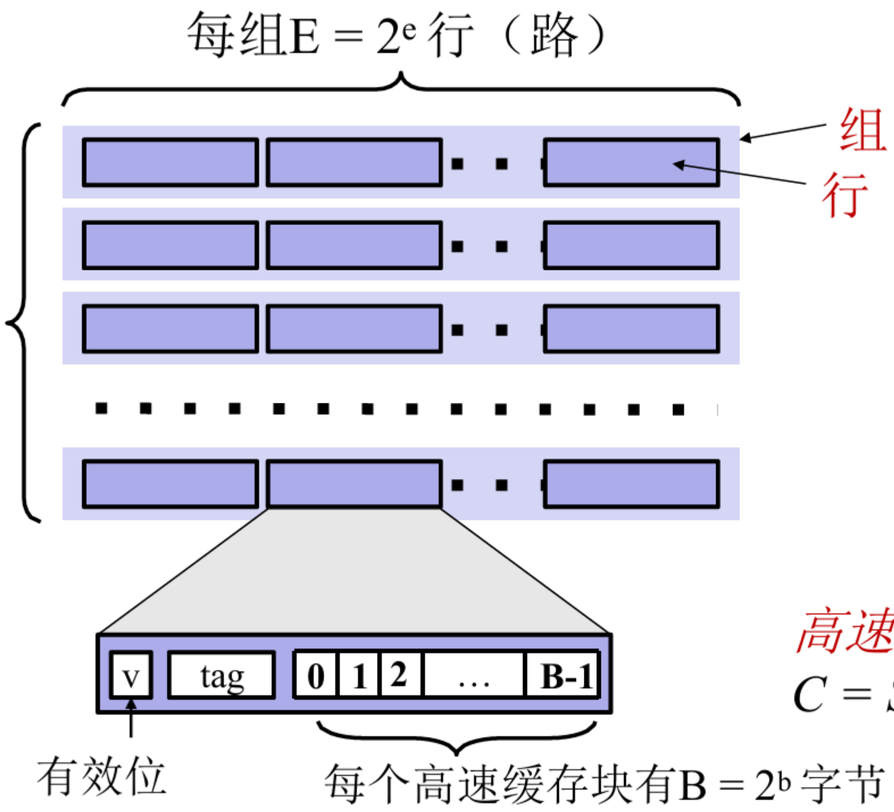
Belady异常：某些页面置换算法，分配工作集增大**可能**PFF反而上升。如FIFO
LRU无此异常

在采用二级页表的分页系统中，CPU页表基址寄存器中的内容是当前进程的一级页表的起始物理地址

合成大虚拟地址：
TLB是组相联，VPN=TLBT+TLBI=页目录号+虚拟页号
TLBT | TLBI | VPO-字节
物理地址：
PPN | PPO-字节

重要澄清：上面的虚拟地址是在管理内存，下面的物理地址结构是在管理缓存
流程上就是上面的过程找到PPN+PPO合成物理地址，再按照下面的划分去比较缓存

高速缓存通用组织结构(S，E，B)
S组，E路，B块大小(字节)
物理地址的另一种划分：
Tag/CT | Index/CI | Offset/CO-字节
直接映射：E=1
全相联：S=1



直接映射到全相联=>
利用率⬆冲突率⬇淘汰算法复杂度⬆
电路复杂度⬆越来越适合小容量cache
全相联无冲突不命中

内存映射
将虚拟内存区域与磁盘上的对象关联起来
内存映射时，只可能是作为共享对象或者私有对象

Fork时若直接将物理内存复制，效率太低
最初fork时，子进程获得父进程虚拟地址空间的完整副本，此时映射的物理内存是相同的，需要写时才复制物理内存，使子进程获得独立的物理内存副本
这叫做COW-CopyOnWrite写时复制

Unit8-I/O与存储

概述

os设备管理的三种方法
操作系统直接操纵设备的运行，例如直接程序控制、中断方式控制
操作系统间接操纵设备的运行，例如DMA和通道方式
操作系统通过使用设备驱动程序以进程形式操纵设备，os只指定标准

os提供统一接口，设备文件存放设备属性
IO系统组成：用户库函数接口：create/remove/open/close/read/write/ioctl
IO内核子系统：IO调度、缓冲、错误处理、保护
设备驱动程序(中断服务程序)
IO硬件

IO向设备发命令过程:
设备控制器包含控制寄存器、数据/状态寄存器等，并维护了不同设备的地址
发命令和相关数据以及读取设备状态 通过 读写设备控制器的寄存器完成

IO控制方式 认为查询不会导致进入阻塞态，而中断和DMA则会

程序直接控制(查询)方式：
发送命令后不断检查状态寄存器看设备是否就绪(命令是否完成)，对于读取指令，确认完成后从数据寄存器取出数据送入内存

中断控制方式：
发送命令并设置允许中断后CPU就不管了，直到设备发送中断信号通知CPU，这时再检查状态寄存器(未完成应该报错)
这里就照应了上下文切换一节。进程(A)等待IO时进入阻塞态，另一个进程(B)运行，收到中断信号就执行中断服务程序(内核级进程)，保护现场，读取数据，唤起阻塞进程(注意，此进程由阻塞态变为了就绪态)并恢复现场继续执行(注意，保护的是B的现场，且通常继续执行B)
弊端：比如读取磁盘数据到内存，这个数据会先进入CPU缓存，再去内存，也就是必须都经过CPU

直接内存存取DMA方式：
中断基础上加入DMA，CPU给DMA发命令并只需要处理DMA完成的中断信号;
DMA给控制器发命令，并负责将数据读到内存，这一步需要暂时挪用CPU对数据地址总线的控制权，送到内存即归还控制权并将字节计数器+1，循环此步直到数据传输完
CPU给DMA的信号:
存放数据的内存起始地址 – DMA控制器的内存地址寄存器;
要输入数据的字节数 – DMA控制器的传送字节数寄存器;
控制字(中断允许、DMA启动位=1) – DMA控制器的控制状态寄存器;
启动位被置1，则启动DMA控制器开始进行数据传输

通道控制方式/远程直接内存存取RDMA方式(RDMA用于智能网卡，功能顾名思义)
通道硬件具有简单CPU功能，流程约等于DMA，CPU每次收到IO请求就形成通道程序放入内存以共通道运行，通道完成任务后发出中断信号给CPU CPU&IO并行工作

IO缓冲技术
硬件缓冲(外设自带的专用寄存器)/软件缓冲(在内存中开辟若干单元作缓冲区)

软件缓冲:
单缓冲：在内存中开辟一个固定大小的区域作为缓冲区
外设和CPU交换数据时，先将被交换的数据写入缓冲区，然后再由需要数据的CPU或外设从缓冲区中取出。
外设与CPU对缓冲区的操作是串行的。
双缓冲：在内存中设置2个大小相同的缓冲区。
外设和CPU可以交替使用这2个缓冲区，一定程度上并行交换数据。
环形缓冲：在内存中设置大小相等的多个缓冲区，并将它们链接称为一个环形链表
三个指针： Head首个缓冲区Full首个存满的缓冲区Empty首个空缓冲区
缓冲池：多个大小相同的缓冲区组成，由系统管理程序统一管理，负责分配、回收工作，可以为更多并行应用提供缓存
三种队列，空缓冲队列，输入缓冲队列，输出缓冲队列
输入设备输入(键盘)：取空缓冲队首，装满放到输入缓冲队尾
输出设备输出(显示屏)：取输出缓冲队首，取完放到空缓冲队尾
进程读：取输入缓冲队首，取完放到空缓冲队尾
进程写：取空缓冲队首，写满放到输出缓冲队尾

相关技术了解:
Linux写磁盘都是先写到buffer中，掉电丢失，使用sync来flushbuffer并写入磁盘/kflush内核线程周期性写磁盘/buffer已满写磁盘；读操作也会使用缓存机制
SPOOLING技术实现独享设备共享，将数据截获到缓冲区，然后共享

用户进程实际分配到的是外存区,即虚拟设备

磁盘
扇区编址问题:
扇区=头+数据区+尾，头尾包含仅磁盘控制器看懂的扇区号码和纠错码信息
盘面H个，磁道C个/盘面，扇区S个/磁道
使用（柱面c，磁头h，扇区s）编址，均从0开始
扇区总数=C*H*S
扇区编号A=c*H*S+h*S+s
已知A，则 s = A%S； h = [A/S]%H； c = [A/(H*S)]

一般数据：盘面512字节，最大数据：H=256,C=1024,S=63
现代使用线性寻址LBA，由地址翻译器来兼容老式3D参数寻址
上面的公式就是LBA和CHS转换

计算:
磁盘访问延迟=(队列时间+控制器时间)+寻道时间+旋转时间+传输时间
寻道时间是磁头移动，会给出
旋转时间=盘片旋转半圈的时间
传输时间=数据量/传输速率

调度算法
计算移动了多少磁道直接移动后位置-移动前位置
先来先服务FCFS：按照请求队列顺序
最短寻道时间优先SSTF：总是选择当前队列中离自己最近的磁道->饥饿问题
扫描/电梯算法SCAN：SSTF基础上，中途不折返，且不论0和最大磁道是否请求，一定要经过才能折返
循环扫描磁盘调度C-SCAN：到达一端后直接返回另一端，期间不处理任何请求，即永远单向扫描，且0和最大磁道也是不论是否有请求都要经过
C-LOOK：C-SCAN基础上，不再必须经过0和最大磁道

IDE硬盘控制器
包含寄存器组，IO端口地址1F0H-1F7H，包含扇区计数器、寻址信息、状态命令寄存器
一般读数据一次性会读几个连续扇区，如4KB

硬盘布局:
第一个扇区-主引导记录MBR-引导可执行代码/存储硬盘最多四个分区信息
这四个分区可能的类型：基本分区/空位(不使用)
基本分区:
每个基本分区的第一个扇区称为分区引导扇区
基本分区的身份额外可能身份：
引导分区：通常设置一个基本分区为引导分区，装载os启动文件
可扩展分区：可以继续划分为多个逻辑分区，从而使得总分区突破四个。其第二个扇区是可扩展分区引导记录

OS启动：
<ROM中的引导程序>加电，CPU执行ROM中的BIOS
<>BIOS硬件自检
<磁盘引导程序>BIOS加载硬盘第一个扇区（MBR）到RAM并执行其中的引导代码
<>MBR引导代码查找活动分区，加载该分区的第一个扇区（分区引导扇区）
<分区引导程序>分区引导扇区代码加载该分区中的Bootloader（如bootmgr、GRUB）
<>Bootloader将操作系统内核加载到RAM，并将控制权交给内核
<操作系统初始化程序>操作系统开始初始化并接管系统

硬盘变为启动盘：

```
┌──────────────────────────────────────┐
│          硬盘（原始状态）              │
├──────────────────────────────────────┤
│ 步骤1：物理格式化（低级格式化）         │
│   ↓ 划分磁道、扇区，建立物理地址        │
│   ↓ 【磁盘扇区的划分在这里完成】        │
├──────────────────────────────────────┤
│ 步骤2：分区                           │
│   ↓ 创建MBR/GPT分区表，划分C、D等逻辑分区 │
│   ↓ 设置活动分区（用于引导）            │
├──────────────────────────────────────┤
│ 步骤3：逻辑格式化（高级格式化）         │
│   ↓ 在每个分区上创建文件系统            │
│   ↓ 创建FAT表、inode表、目录结构等      │
│   ↓ 【文件系统根目录的建立在这里完成】   │
├──────────────────────────────────────┤
│ 步骤4：安装操作系统                    │
│   ↓ 复制系统文件到已格式化的分区        │
│   ↓ 安装引导程序（bootloader）到MBR和引导分区 │
│   ↓ 配置启动信息                      │
└──────────────────────────────────────┘
```

前面的交换分区问题:
换下来的页直接存入扇区，通过扇区号寻址，这样使用的分区称为交换分区，这样使用的磁盘称为生磁盘，交换分区通常两倍内存大小

SSD
体系结构：NAND FLASH存储介质/嵌入式控制器/RAM缓存
顺序访问快于随机访问，随机写慢，擦除数据块较慢
比磁盘更快，能耗更低，更结实

FLASH:一个晶圆2GB/一个组1GB/一个块512KB(擦除单元)/一个页4KB;

SSD软件的两个主要模块:
数据缓存管理模块
闪存翻译层模块FTL(假装自己是HDD从而直接替代HDD):
地址映射：<逻辑扇区号LSN，几个扇区size>=><package，die，plane，block，page>
损耗平衡:
动态：在请求到达时，选取擦除次数较少的块作为请求的物理地址
静态：将冷数据尽可能放到擦除次数多的块上，热数据相反
垃圾回收：清除使用过程中产生的失效页

Unit9文件系统

文件的物理结构:
物理盘块连续存储：将文件的起始盘块和盘块个数存放在文件描述信息节点中(FCB或File Header)
链式存储：File Header磁盘块起始地址位置，每个磁盘块都记录下一个磁盘块位置
显式：如FAT使用一张表；隐式：指针就在数据块中包含
索引存储：File Header记录索引块位置，索引块中有索引表记录所有磁盘块地址
多级索引之一级索引：文件头的一组表项记录的是第一级索引表块的块号，第一级索引表块中的索引表项记录文件逻辑记录所在的磁盘块号

文件的逻辑结构:
使用文件系统管理。逻辑结构使用树状目录

文件被打开后，用户对文件的任何操作都只须使用fd而非路径名
整个系统只有一个系统打开文件表，同一个文件打开多次只需改变引用计数

目录：描述文件的集合
设计思想：将磁盘块分为几部分，一部分专门存放文件头，一部分专门存放数据。
查找时，通过目录的文件头找到其数据，其数据中存放类似<名称，文件头数组索引号>的内容，由此找到文件头，再找到数据

UNIX分区的基本结构
引导块 | 超级块 | 索引结点数组 | 数据块
引导块存放引导OS信息，若该分区没有OS则为空;
超级块记录分区块数，块大小，空闲块数量、指针，空闲文件头数量、指针等
索引节点数组存放所有文件的文件头，其中UNIXroot目录索引节点号为2

管理空闲块:

空闲位图/位向量

0011110011101001表示磁盘块2,3,4,5,8,9,10,12已被占用(1表示占用，从块0开始)

可快速分配盘块但是位向量很大

空闲链表

空闲块头指针开始，每个空闲块指向下一个空闲块

文件系统考虑因素-提升速度

磁盘缓存：在内存中缓存少部分磁盘盘块。Hashing组织缓存池，LRU替换，写直达和写回两种策略

内存中常驻部分文件的FCB，如根目录，当前目录

-提供保护

r/w/x权限放进文件头中，进程发起访问请求，其PCB中带有用户信息(uid，gid)，文件访问时校验权限

-提供容错-RAID

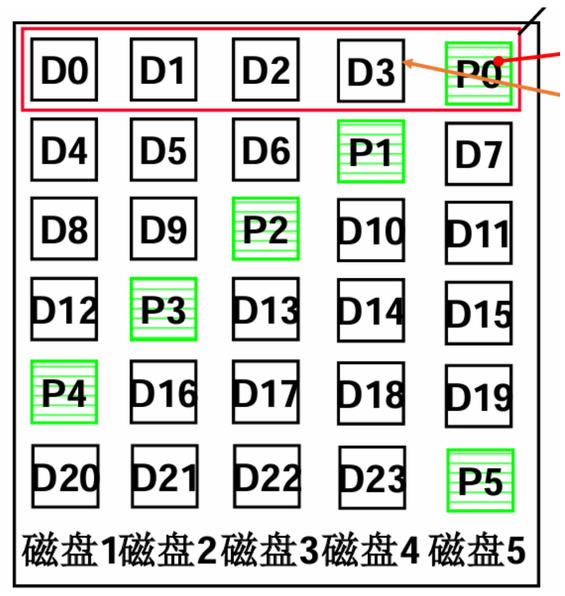
基本思想是冗余

RAID1：每次都写到两个磁盘上，从磁盘A读，出错则转向B，然后将数据拷贝回A

RAID5+:

每一行校验位Px等于同行其余位异或，如P0=D0 xor D1 xor D2 xor D3

恢复时，等于同行其他数据异或，如D3=D0 xor D1 xor D2 xor P0



EXT2文件系统

磁盘结构:

引导块 | 块组0 | 块组1...

块组结构(块组除了最后一块等大):

超级块(1) | 组描述符(n) | 块位图(1) | 索引节点位图(1) | 索引节点表(n) | 数据区(n)

启动块大小确定，存储磁盘分区信息和启动信息，任何文件系统都不能使用启动块。启动块之后才是ext2文件系统的开始

超级块：描述整个分区的文件系统信息，如块大小、版本号、上次mount时间等;

一般不同的块组的超级块信息是一样的，可能会在系统挂载并活跃时不同，但最终会被同步

块组描述符：描述一个磁盘块组的相关信息，若干块组描述符描述了所有块组的属性

两个位图：同本页开始时位图描述，1表示占用。因此磁盘块大小决定了最大数据块个数和最大索引结点块个数

索引节点：一个inode占用128字节

关于索引，一个文件能且只能对应一个inode，inode中使用组合索引，12个直接索引，一二三级间接索引各一个

直接索引能索引12个数据块

块大小4KB，索引表项块号信息4B，则一个一级间接索引能索引1024个数据块

目录文件：
目录文件的数据块存储的是目录项；
一个目录项包含该文件的文件名、文件类型(0~7)、文件的inode号(0表示未使用)等信息
文件类型：未知(0)，普通文件(1)，目录(2)等等

Windows-FAT
组织形式：
一个簇即为一组连续扇区的集合，是FAT分配的基本单位。文件存储在多个簇中。一个簇的簇链表示下一个簇
FAT卷结构：
引导区 | 文件分配表FAT1 | 文件分配表FAT2 | 根目录 | 其他目录和文件
引导区：可能存在引导代码，同时记录扇区字节数，簇扇区数，根目录起始位置，文件系统类型等各种信息
FAT1：每一项对应一个簇，包含信息：是否空闲(等于0空闲)/簇链/链尾标志信息(0xFFF/0xFFFF/0xFFFFFFFF)
FAT2：对FAT1的完全镜像备份
根目录：保存该卷根目录项，目录项包含目录文件起始簇号。文件名遵守8.3规则则目录项大小32字节，否则长文件名要通过增加多个目录项表示。FAT12/16预留256个空间，而32无限制
其他目录和文件：存放文件数据(目录也是文件，数据即存放目录项)

FAT12最大簇数 = 2^12个 = 4K个
规定簇大小 = 0.5KB ~ 8KB
FAT12卷大小≤32MB
FAT16最大簇数 = 2^16个 = 64K个
规定簇大小 = 0.5KB ~ 64KB
FAT16卷大小≤4GB
FAT32最大簇数 = 2^32个 = 4G个
限定使用2^28个 = 0.25G个
规定簇大小 = 4KB ~ 32KB
FAT32卷大小≤1TG~8TB

FAT16/FAT32限制单个文件最大为4GB

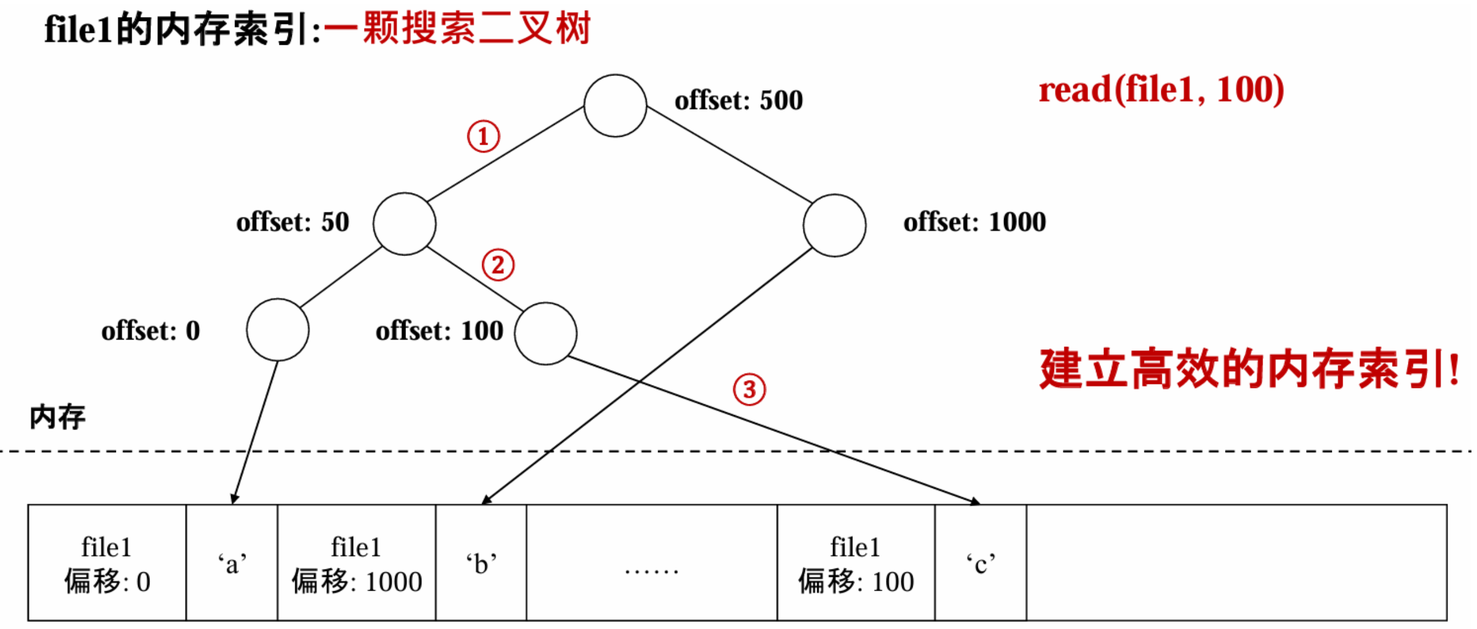
最简化计算时，FAT表项可以认为只存储下一个块号，因此假设512K块，至少用19bit，如果半字节对齐，那么对齐到20bit，即一个表项占2.5Byte

Journal模块简单运作：
两个指针：head&tail，初始指向同一位置
写数据先写到head处，然后将尾指针移动到写的数据末尾，然后再将数据写到磁盘中并更新位图，然后移动头指针到尾指针处，这一步叫做提交journal
崩溃后，扫描head到tail看有没有留存的有效数据，若有就重写这部分数据并更新位图

LFS日志型文件系统-解决随机写慢的问题

顺序写入文件，每个文件包含头和数据两部分，头存储文件名和偏移

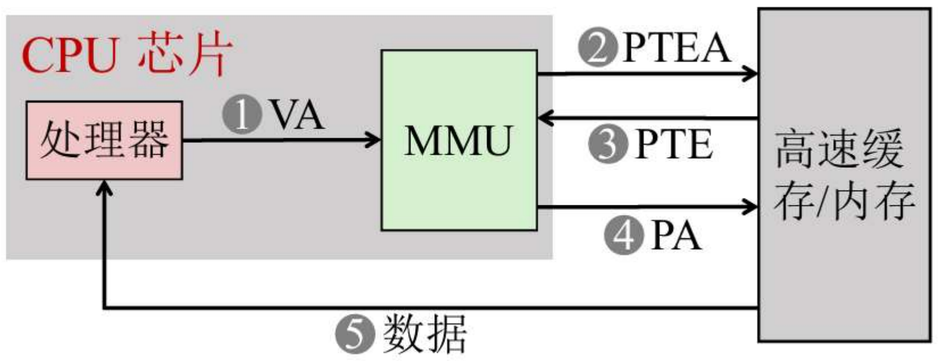
查找使用搜索二叉树



写满要回收垃圾时，扫描整个日志，收集有效数据从头开始写，以获得大段连续空间

F2FS对flash友好型文件系统，采用了基于日志追加写 数据冷热分离 、区分顺序写和随机写 的数据布局等技术，大幅提升了随机写性能

地址翻译流程图大全



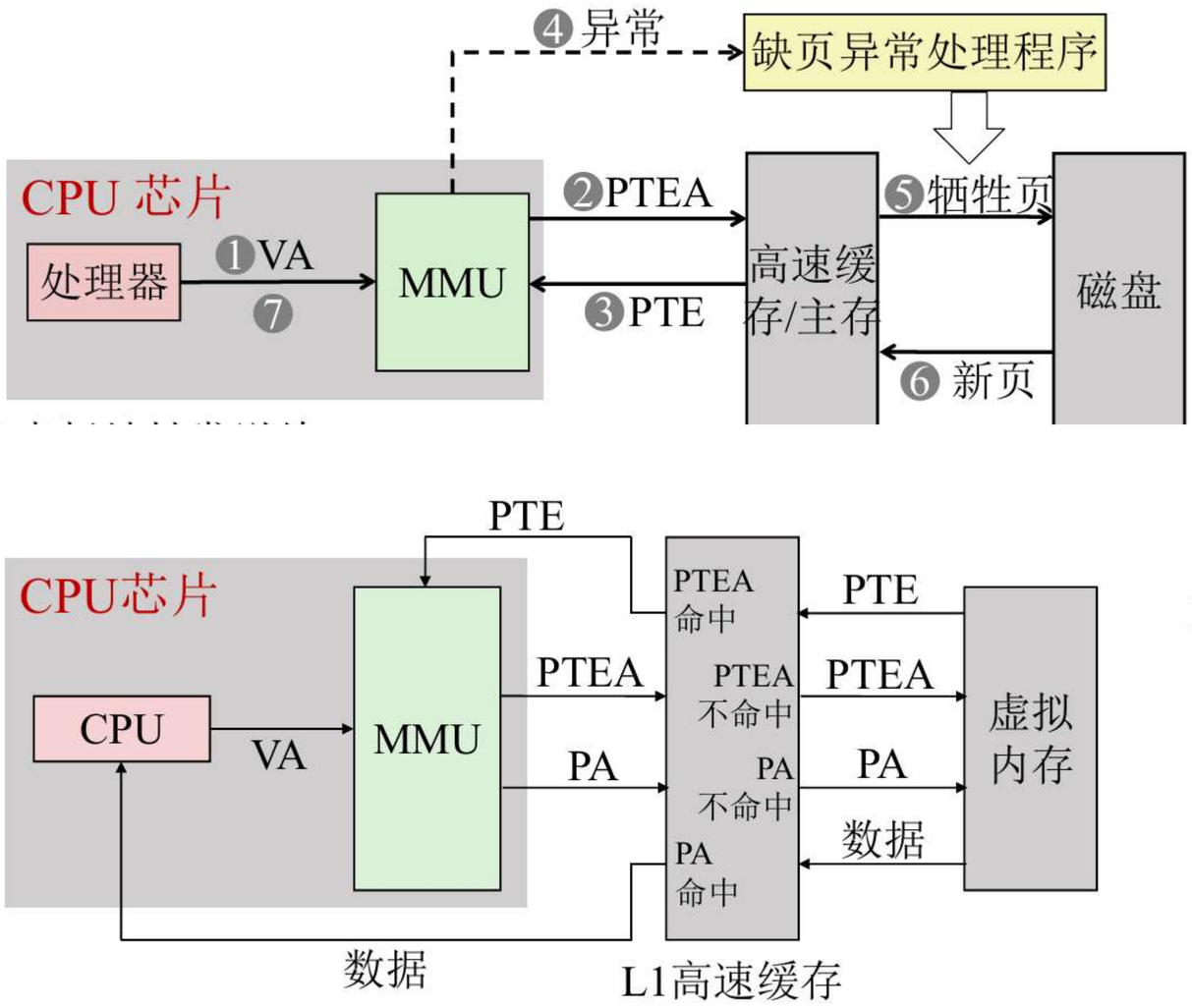
VA：virtual address虚拟地址

PA：physical address物理地址

PTE：page table entry页表条目

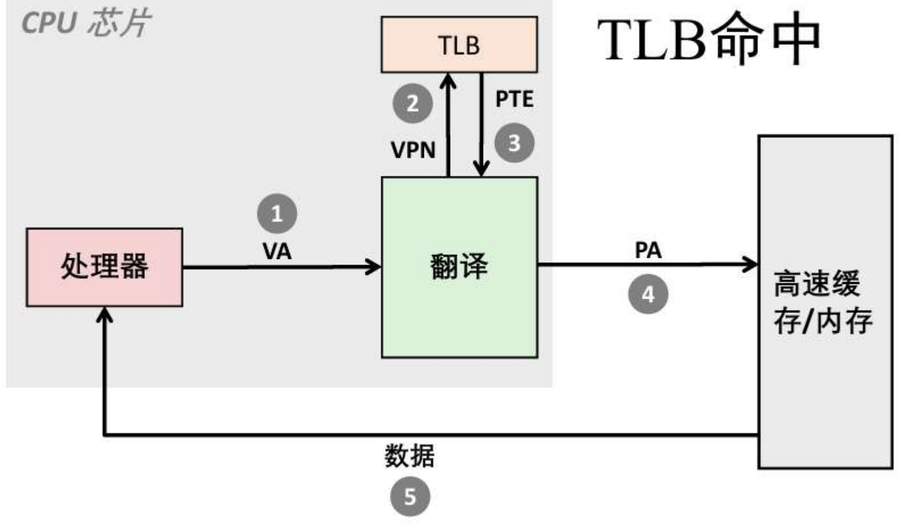
PTEA：PTE address页表条目地址

地址翻译：页面命中

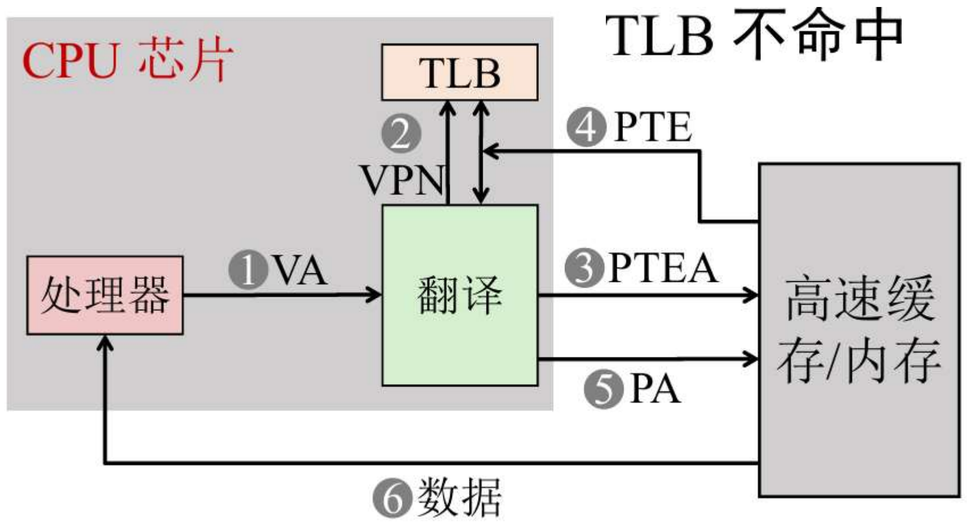


地址翻译：缺页异常

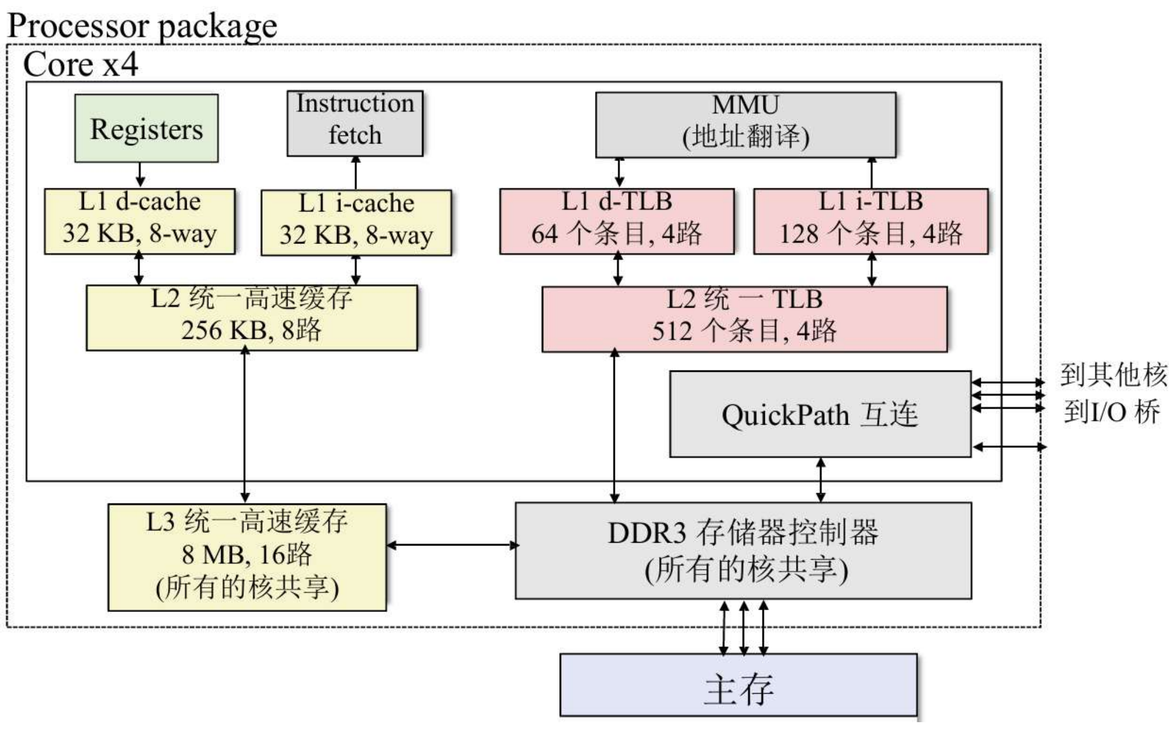
结合高速缓存和虚拟内存

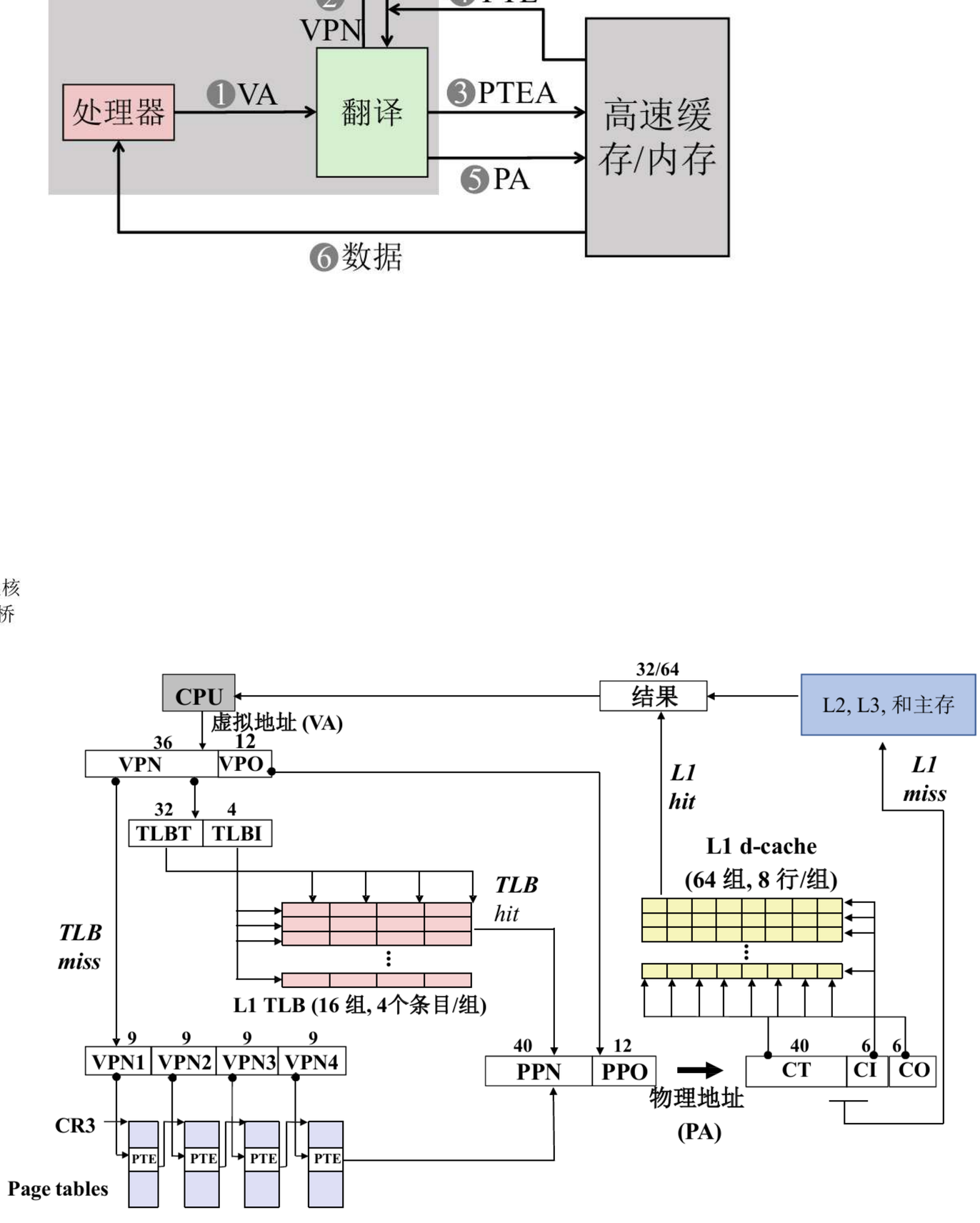


TLB命中



TLB 不命中

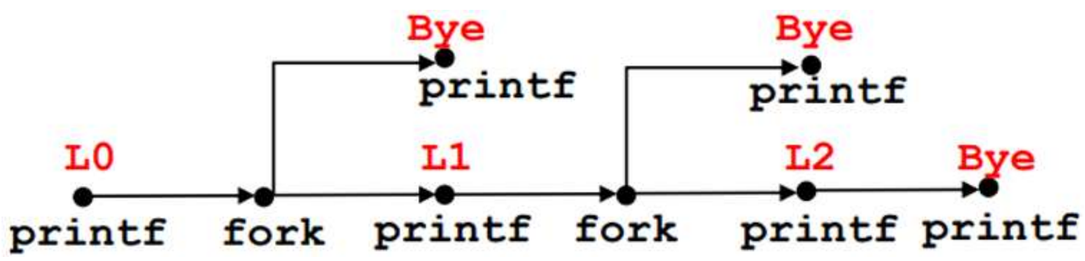




补充阅读
操作系统如何执行a.out?
execve函数在当前进程中加载并运行新程序 a.out的步骤:
代码和始化数据映射到.text和.data区
.bss和栈映射到匿名文件（通过匿名内 存映射完成）
进行动态/静态链接库映射 (通过内存 映射完成)
设置PC，指向代码区域 的入口点
Linux根据需要换入代码和数据页面

题目小点记录:
1.管道是半双工，不允许双向数据传输；允许多个互斥读写；其大小是固定的，如linux是4KB
2.

```
void fork4()
{
    printf("L0\n");
    if (fork() == 0) {
        printf("L1\n");
        if (fork() == 0) {
            printf("L2\n");
      }
    }
    printf("Bye\n");
}
```



3.在用户级线程中，有关线程管理的所有工作如线程切换都由应用程序完成，无须内核的干预，内核意识不到线程的存在
4.时钟中断发生后，系统会修改当前的进程在时间片内的剩余时间
5.在同一进程中的两个线程之间进行上下文切换时，以下哪些内容必需保存和 恢复。
①
①计算寄存器和栈指针寄存器 ②页表根指针 ③使用fopen打开的流中的缓冲数据 ④文件描述符表的内容
6.